網路技術與應用
Programming Assignment 3
安全通訊 **Client&Server**

B10801011 張鈞傑

1. 編譯與執行

- 編譯指令:
  - g++ server.cpp -o server -lssl -lcrypto -pthread
  - g++ client.cpp -o client -lssl -lcrypto
- 執行指令:
  - 啟動伺服器端: ./server
  - 啟動客戶端: ./client

2. 程式結構與功能

2.1 客戶端 (client.cpp)

- 主要功能:
  - 與伺服器建立加密連線。
  - 提供用戶登入、註冊和點對點交易功能。
  - 接收伺服器回應, 並根據用戶選項執行操作。
- 核心程式碼與功能說明:
  - Socket 初始化與加密連線 (main 函式):

```
int sockfd = socket(AF_INET, SOCK_STREAM, 0);

if (sockfd == -1) {

    perror("Socket creation failed");

    exit(EXIT_FAILURE);

}

SSL_CTX *ctx = SSL_CTX_new(TLS_client_method());

SSL *ssl = SSL_new(ctx);

SSL_set_fd(ssl, sockfd);

if (SSL_connect(ssl) == -1) {

    perror("SSL connection failed");

    exit(EXIT_FAILURE);

}
```

- 解釋:
  - 創建 TCP Socket，並設定為加密模式。
  - 使用 OpenSSL 的 SSL_connect() 方法與伺服器建立 TLS 連線。
- 用戶指令處理與伺服器互動:

```
while (true) {
    std::string input;
    std::cin >> input;
    SSL_write(ssl, input.c_str(), input.length());
    char buffer[1024];
    SSL_read(ssl, buffer, sizeof(buffer));
    std::cout << "Server response: " << buffer << std::endl;
}
```

- 解釋:
  - 循環接收用戶輸入，通過 SSL 發送給伺服器。
  - 接收伺服器的加密回應，並解密後顯示給用戶。
- 點對點交易功能：當用戶選擇 P2P 傳輸時，程式將額外創建一個 Socket 與其他客戶端直接進行通信，並利用 SSL 保證安全。

2.2 伺服器端 (server.cpp)

- 主要功能:
  - 使用線程池處理多個客戶端連接。
  - 管理用戶資訊，支援註冊、登入、查詢和交易等功能。
  - 實現 P2P 資料傳輸，並通過 SSL 確保數據安全。
  - 核心程式碼與功能說明:
- 伺服器初始化與監聽 (main 函式):

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);
bind(server_fd, (struct sockaddr *)&address, sizeof(address));
listen(server_fd, 10);
```

- 解釋:
  - 創建伺服器的 TCP Socket，並設定為監聽模式。
  - bind() 綁定本地地址與端口，listen() 開始等待客戶端連接。
  - 加密連線與線程分配:

```
SSL_CTX *ctx = SSL_CTX_new(TLS_server_method());
while (true) {
    int client_fd = accept(server_fd, (struct sockaddr *)&address, &addrlen);
    SSL *ssl = SSL_new(ctx);
    SSL_set_fd(ssl, client_fd);
    if (SSL_accept(ssl) == -1) {
        perror("SSL handshake failed");
        continue;
    }
    thread_pool.addTask([ssl]() { handleClient(ssl); });
}
```

- 解釋:
  - 每當接收到新的客戶端連接，伺服器會為其建立 SSL 加密通道。
  - 通過線程池分配一個線程處理該客戶端的請求。
- 用戶資料管理 (UserList 類):

```
bool UserList::registerUser(const std::string &username, const std::string
&password) {
    if (findUser(username)) return false;
    users.push_back(User(username, password));
    return true;
}
```

- 解釋:
  - 管理用戶註冊、登入等功能，確保用戶名唯一性。
  - 使用內存結構（如 vector）保存用戶數據。
- 交易邏輯 (transact 方法) :

```
bool transact(const std::string &fromUser, const std::string &toUser, int amount) {
    User *sender = findUser(fromUser);
    User *receiver = findUser(toUser);
    if (sender->balance < amount) return false;
    sender->balance -= amount;
    receiver->balance += amount;
    return true;
}
```

- 解釋:
  - 確保用戶餘額足夠，完成交易後更新雙方的餘額。
  - 返回交易結果給請求的客戶端。

3. 程式執行環境說明

- 環境需求:
  - 作業系統:Linux/Unix
  - 編譯器:g++ (支援 C++11 或更高版本)
  - 第三方庫:OpenSSL (libssl 和 libcrypto)
  - 支援多線程的環境 (如 pthread)
- 運行步驟:
  - 啟動伺服器後，客戶端可以通過指定的 IP 和 Port 與伺服器通信。
  - 客戶端輸入命令如 “register” 或 “login” 開始與伺服器互動。

4. 資料傳輸與安全說明

- SSL/TLS 加密:
  - 使用 OpenSSL 庫對數據進行加密，保護通信內容。
  - 客戶端與伺服器在建立連線時執行 SSL 握手，確保安全通道。
- 多線程處理:
  - 伺服器使用線程池並發處理客戶端請求，避免阻塞。
- 資料傳輸邏輯:
  - 客戶端發送指令（如登入、查詢），伺服器根據指令執行相應操作並回應。